Universidade de Coimbra
Faculdade de Ciências e Tecnologia
Departamento de Engenharia Informática

Integração de Sistemas

Projeto 2 - Enterprise Application Integration

Docente

Pedro Nuno San-Bento Furtado

Equipa

Carlos João Lopes Mendes, nº2017257272

Filipe Miguel Fonseca dos Santos, nº2017271196

Índice

Autoavaliação do projeto 3
Entities 5
Loader - Alínea a) 6
EJB (com integração de JPA) - Alínea b), c) e d) 7
Serviços Web SOAP - Alínea b) e c) e f) 8
Serviços REST – Alínea d) 9
Cliente – alínea e) 10

Autoavaliação do projeto

a. Lists of what the group succeeded to do and what is missing

-O grupo conseguiu atingir os objetivos básicos que eram pedidos no enunciado, não havendo em princípio nenhum aspeto que esteja em falta.

b. self-evaluation of the group (0-100%)

-90%

c. List of what each student contributed (no repetitions)

-Para o nosso projeto, decidimos realizar todo o trabalho em conjunto, através de videochamadas nas quais um elemento partilhou o ecrã e o outro auxiliou a pensar na solução e a encontrar erros. Temos os tempos de chamada registados e podemos associá-los ao que fizemos em cada uma delas, mencionando quem nessa chamada partilhou o ecrã e quem ficou a auxiliar. Devido ao IDE do eclipse ter uma má performance na máquina de um dos elementos do grupo, de forma a não atrasar o trabalho, todas as partes relacionadas com a utilização do eclipse IDE foram feitas pelo mesmo elemento a partilhar o ecrã.

Chamada	Tempo	Tema
1	2h e 4min	Criação de tabelas com informação do researchGate e com base num esquema relacional
2	1h e 30 min	Continuação da parte anterior
3	2h 30 min	Criação das diferentes entidades
4	1h 22 min	Continuação da parte anterior
5	52 min	Melhoria no Loader
6	2 h e 28 min	Trabalho no Loader, JPA e EJB
7	1h 16 min	Continuação no EJB
8	2h e 32 min	Continuação no EJB e início dos Web Services SOAP
9	2h 26 min	Continuação
10	5h e 29 min	Continuação do SOAP
11	2h e 34 min	Continuação do SOAP e do WebFrontEnd
12	1h e 18 min	Continuação
13	1h e 22 min	Continuação
14	1h e 59 min	Alteração das entidades para aceitar anotações de relações (ex: manyToMany)
15	1h e 28min	Continuação
16	1h e 28 min	Continuação do SOAP (Conclusão ponto b e c)
17	2h e 3 min	Inicio do REST
18	2h e 42 min	Ponto f a funcionar e continuação do REST
19	3h e 15 min	Continuação do REST
20	3h e 20 min	Continuação do REST
21	1h e 4 min	Conclusão do REST e continuação do Cliente
22	3h e 25 min	Alteração do Loader para usar JPA
23	2h e 25 min	Relatório
Dúvidas com o professor		
1	1h e 10 min	Integração de JPA e EJB
2	32 min	Aceder à response do REST
3	2h e 20	Aceder à response do REST

d. self-evaluation of each student in the group (0-100%)

Carlos Mendes: 90%
Filipe Santos: 90%

e. hours of effort by each student separately

Neste caso, como em vez de dividirmos tarefas de forma autónoma, fizemos tudo em conjunto, as horas de esforço são as mesmas.

Horas de esforço = ~55 horas e 5 minutos

Entities

Para este projeto utilizámos 4 entidades que refletem a informação que necessitamos de apresentar e se inserem nos projetos de Loader e EJB.

Relativamente às relações entre entidades, utilizámos o seguinte esquema.

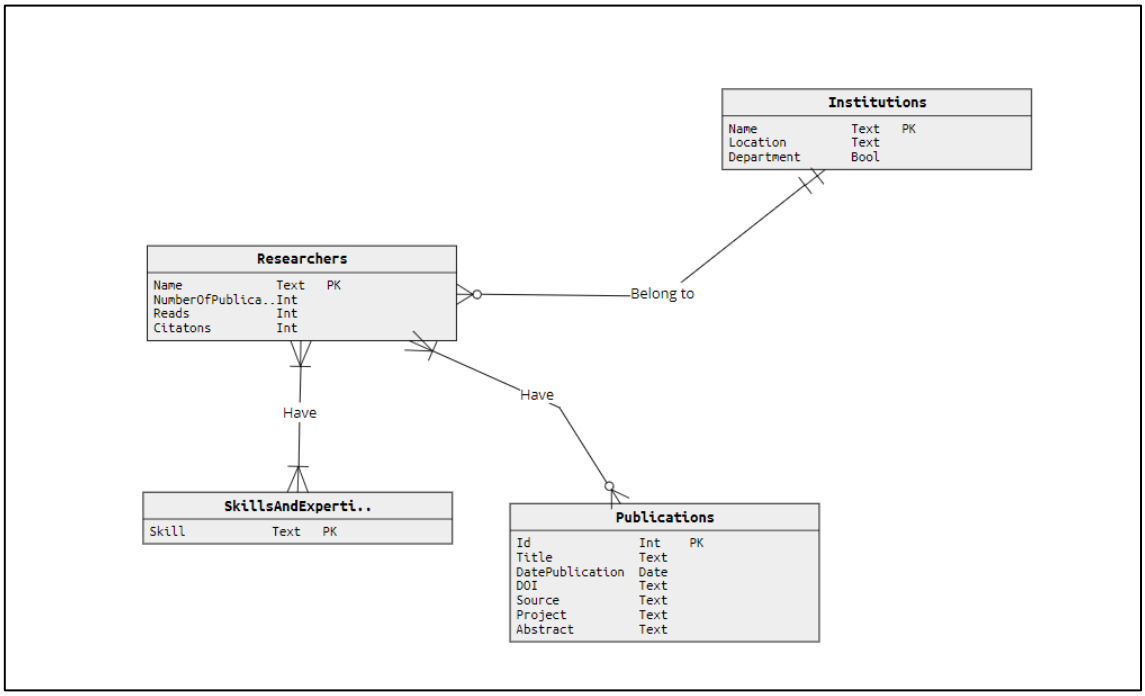

Figura 1: esquema relacional da base de dados

Nas entities, utilizámos as anotações de persistence para estabelecer as diferentes relações.

```
@ManyToMany
@JoinTable(
  name = "researchers_skillsandexpertise",
  joinColumns = @JoinColumn(name = "researchers_name"),
  inverseJoinColumns = @JoinColumn(name = "skillsandexpertise_skill"))
private Set<SkillsAndExpertise> skills = new HashSet<>();
```

Figura 2: Anotações de persistence usadas na classe dos researchers (Owning side)

```
@ManyToMany(mappedBy = "skills")
private Set<Researchers> researchers = new HashSet<>();
```

Figura 3: Anotações de persistence usadas na classe das skills

Loader - Alínea a)

A aplicação Loader trata-se de uma standalone cuja funcionalidade é manipular os dados de uma base de dados. Estes dados estão hardcoded dentro das diferentes classes sendo que basta correr a função main de cada classe.

Esta aplicação tem 3 classes diferentes:

- LoadDataJPA: Adiciona os dados à base de dados através de JPA para os persistir. As tabelas são criadas automaticamente também pelo JPA.

```java
public class LoadDataJPA {

    public static void main(String[] args) {
        EntityManagerFactory emf = Persistence.createEntityManagerFactory("TestPersistence");
        EntityManager em = emf.createEntityManager();

        Institutions iNew1 = new Institutions("Federal University Otuoke", "Bayelsa-Nigeria", "Biochemistry", null);
        em.getTransaction().begin();
        em.persist(iNew1);

        em.getTransaction().commit();

        System.out.println("END");

        // Close an application-managed entity manager.
        em.close();
        // Close the factory, releasing any resources that it holds.
        emf.close();
    }

}
```

Figura 4: Exemplo de persistir um objeto através de JPA

- LoadDataSQL: Semelhante ao anterior, este usa comando de SQL para atingir o mesmo fim, sem utilização de JPA. Este foi feito inicialmente para testar as várias funcionalidades.
- DeleteData: Utiliza comandos SQL para apagar as tabelas e respetivos dados criados pelas classes anteriores.

De forma a que funcione corretamente, este projeto também tem as classes que servem como entities nele.

EJB (com integração de JPA) - Alínea b), c) e d)

Este projeto é aquele que contém os session beans que são usados para ir buscar informação à base de dados e depois enviá-la aos diferentes serviços web que chamam esses mesmos beans. Cada serviço tem o seu próprio bean associado, para garantir independência entre as diversas funiconalidades.

```
@PersistenceContext(name = "TestPersistence")
private EntityManager em;
public ArrayList<InstitutionsCopy> readInstitutions() {
    String jpql = "SELECT i FROM Institutions i";
    TypedQuery<Institutions> typedQuery = em.createQuery(jpql, Institutions.class);
    List<Institutions> mylist = typedQuery.getResultList();

    ArrayList<InstitutionsCopy> isCopy = new ArrayList<>();
    Set<Researchers> researchers = new HashSet<>();
    System.out.println("CREATING COPY");
    for(Institutions ins : mylist) {
        InstitutionsCopy iCopy = new InstitutionsCopy();
        iCopy.setName(ins.getName());
        iCopy.setLocation(ins.getLocation());
        iCopy.setDepartment(ins.getDepartment());
        researchers = ins.getResearchers();
        for(Researchers rs : researchers) {
            iCopy.getResearchers().add(rs.getName());
        }
        isCopy.add(iCopy);
    }

    return isCopy;
}
```

Figura 5: Exemplo de um método que vai buscar informação à base de dados (vai buscar a informação relativa às instituições)

Cada bean tem também associada uma interface remota para os métodos serem usados pelos serviços web. Além disso, tratam também do business logic necessário para mandar os dados aos serviços de forma adequada.

Serviços Web SOAP - Alínea b) e c) e f)

Este projeto trata-se de um dynamic web project que tem duas classes, uma associada com researchers e outra com publications. Estas classes tem a função de chamar os diferentes métodos do projeto EJB e de mandar a informação necessária para os clientes.

```
public String[] getPublications() {
    Context context;

    Properties jndiProperties = new Properties();

    jndiProperties.setProperty("java.naming.factory.initial", "org.jboss.naming.remote.client.InitialContextFactory");
    jndiProperties.setProperty("java.naming.provider.url","http-remoting://localhost:8080");
    jndiProperties.setProperty("jboss.naming.client.ejb.context","true");

    try {

        context = new InitialContext(jndiProperties);
        PublicationsBeanRemote myejb = (PublicationsBeanRemote) context.lookup("Project_EAR/Project_EJB/PublicationsBean


        Object[] oa = myejb.readPublications().toArray();

        String[] stringArray = Arrays.copyOf(oa, oa.length, String[].class);

        return stringArray;

    } catch (NamingException e) {
        e.printStackTrace();
    }
    return null;
}
```

Figura 6: Exemplo de um método que serve de base para um método do serviço SOAP

Estas classes são usadas para criar serviços web através das funcionalidades do eclipse IDE. Num dos projetos criados desta maneira, adicionámos um ficheiro jsp para ir buscar informação sobre as publicações suportada com web front end (alínea f).

```
<jsp:useBean id="samplePublicationsSoapProxyid" scope="session"
    class="service.PublicationsSoapProxy" />
<%
    String[] s = samplePublicationsSoapProxyid.getPublications();
    ArrayList<String> wordList = new ArrayList<>();
    wordList = new ArrayList<String>(Arrays.asList(s));
    int i = 0;
    int j = 0;
    int numberResearchers;
%>
  <%  for(i = 0; i < wordList.size(); ++i) { %>
            <h2>Publication <%=wordList.get(i)%> <%++i;%> </h2>
            <br>
            <b>Title:</b> <%=wordList.get(i)%> <%++i;%>
            <br>
            <b>Date:</b> <%=wordList.get(i)%> <%++i;%>
            <br>
            <b>DOI:</b> <%=wordList.get(i)%> <%++i;%>
            <br>
            <b>Source:</b> <%=wordList.get(i)%> <%++i;%>
            <br>
```

Figura 7: Excerto de um jsp que utiliza um serviço SOAP.

Serviços REST – Alínea d)

Os métodos usados no serviço são semelhantes aos usado nos serviços SOAP, sendo que adicionámos as anotações necessárias para enviar os dados através de "post" em formato JSON.

```
@POST
@Path("/post1")
@Consumes(MediaType.APPLICATION_JSON)
@Produces(MediaType.APPLICATION_JSON)
public Response getInstitutions(String s) {
    Context context;

    Properties jndiProperties = new Properties();

    jndiProperties.setProperty("java.naming.factory.initial", "org.jboss.naming.remote.client.InitialContextFactory");
    jndiProperties.setProperty("java.naming.provider.url","http-remoting://localhost:8080");
    jndiProperties.setProperty("jboss.naming.client.ejb.context","true");

    try {
        context = new InitialContext(jndiProperties);

        InstitutionsBeanRemote myejb = (InstitutionsBeanRemote) context.lookup("Project_EAR/Project_EJB/InstitutionsBea

        ArrayList<InstitutionsCopy> is = myejb.readInstitutions();

        InstitutionsList isList = new InstitutionsList(is);

        return Response.status(Status.OK).entity(isList).build();

    } catch (NamingException e) {
        e.printStackTrace();
    }

    return null;
}
```

Figura 8: Exemplo de um método de REST

Na integração com REST com beans e JPA, tivemos algumas dificuldades na transmissão de dados para o cliente.

O problema tratava-se na transmissão de objectos JSON criados a partir de classes que utilizavam anotações de JPA, pois a criação não era feita corretamente. Para ultrapassar esta dificuldade, optámos por criar uma classe idêntica à entidade necessária (institutions), mas que não recorria ao JPA e foi a partir dessa nova classe que criámos os objetos JSON. A transição entre as classes é feita no bean respetivo.

Cliente – alínea e)

O projeto cliente trata-se de um simples projeto java que tem como função atuar como um cliente que chama os métodos mencionados nas alíneas b), c) e d) através da linha de comandos.

Nos métodos que recorriam a REST, tivemos algumas dificuldades pois estávamos a tentar deserializar objetos ArrayList de institutions, mas como o ArrayList não se trata de uma classe genérica, isso trazia problemas. A nossa solução passou por criar uma classe cuja única função é conter um ArrayList com as institutions, de forma a termos uma classe genérica.

```
public void showPublications() throws RemoteException {
    System.out.println("");
    PublicationsSoapProxy proxy = new PublicationsSoapProxy(
            "http://localhost:8080/Project_SOAP_Web_Services/services/PublicationsSoap");
    String[] s = proxy.getPublications();
    ArrayList<String> wordList = new ArrayList<>();
    wordList = new ArrayList<String>(Arrays.asList(s));
    int i, j;
    int numberResearchers;
    for (i = 0; i < wordList.size(); ++i) {
        System.out.println("Publication " + wordList.get(i));
        ++i;
        System.out.println("\tTitle: " + wordList.get(i));
        ++i;
        System.out.println("\tDate: " + wordList.get(i));
        ++i;
        System.out.println("\tDOI: " + wordList.get(i));
        ++i;
        System.out.println("\tSource: " + wordList.get(i));
        ++i;
        System.out.println("\tProject: " + wordList.get(i));
        ++i;
        System.out.println("\tAbstract: " + wordList.get(i));
        ++i;
        numberResearchers = Integer.parseInt(wordList.get(i));
        ++i;
        System.out.printf("\tResearchers: ");
        for(j = 0; j < numberResearchers; ++j, ++i) {
            System.out.printf("%s; ", wordList.get(i));
        }
```

Figura 9: Exemplo de um método do cliente que usa um serviço SOAP

A chamada ao serviço SOAP é feita através da classe proxy respetiva, que foi gerada automaticamente quando criámos os serviços web por meio do IDE, que mencionámos anteriormente.

```java
public void showInstitutions() {
    Client client = ClientBuilder.newClient();
    //client.register(JacksonJaxbJsonProvider.class);
    WebTarget target = client.target("http://localhost:8080/Project_REST/rest/institution/post1");
    Response response;

    InstitutionsList isList = new InstitutionsList();
    Entity<InstitutionsList> input = Entity.entity(isList, MediaType.APPLICATION_JSON);
    response = target.request().post(input);

    isList = response.readEntity(InstitutionsList.class);
    ArrayList<InstitutionsCopy> is = isList.getInstitutionsList();

    int i = 0, j;
    for(i = 0; i < is.size(); ++i) {
        System.out.println("Institution " + i);
        System.out.println("\tName: " + is.get(i).getName());
        System.out.println("\tLocation: " + is.get(i).getLocation());
        System.out.println("\tDepartment: " + is.get(i).getDepartment());
        System.out.println("");
        System.out.printf("\tResearcher(s): ");
        ArrayList<String> rs = is.get(i).getResearchers();
        for(j = 0; j < rs.size(); ++j) {
            System.out.printf("%s; ", rs.get(j));
        }
        System.out.println("\n");
    }

    response.close();
}
```

Figura 10: Exemplo de um método do cliente que usa um serviço REST

Os serviços REST são chamados por meio do endereço que é definido no projeto REST em sim (na configuração e no método em si).